

Ciencia de Datos



Unidad 4: Aprendizaje Supervisado

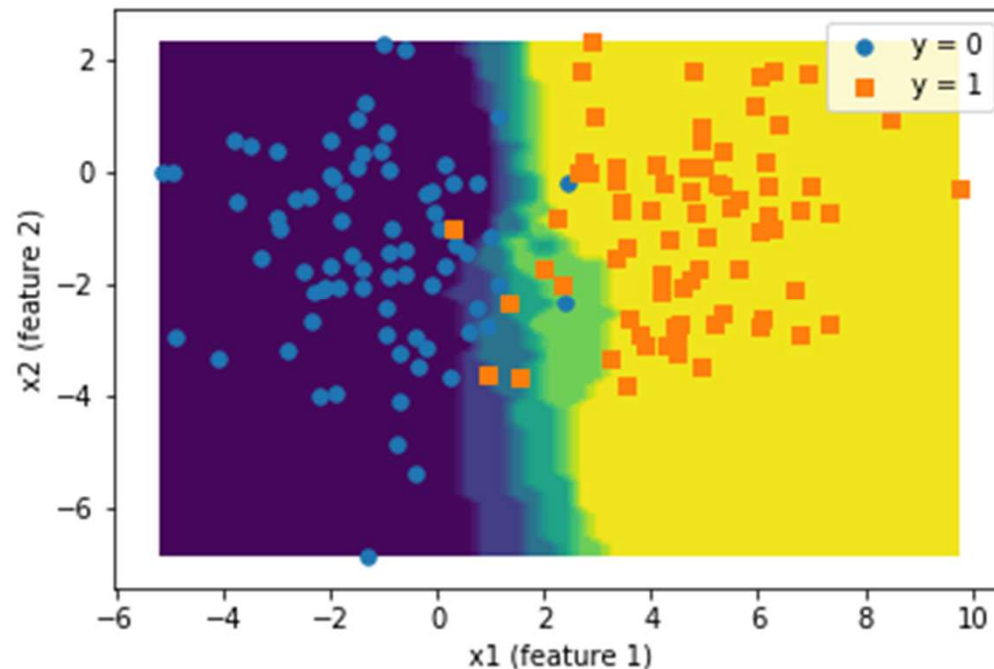
Tema: KNN

Mgtr. Ing. Mariano Martín Gualpa (mgualpa@frc.utn.edu.ar)

K Nearest Neighbours (KNN)

K Nearest Neighbours (KNN)

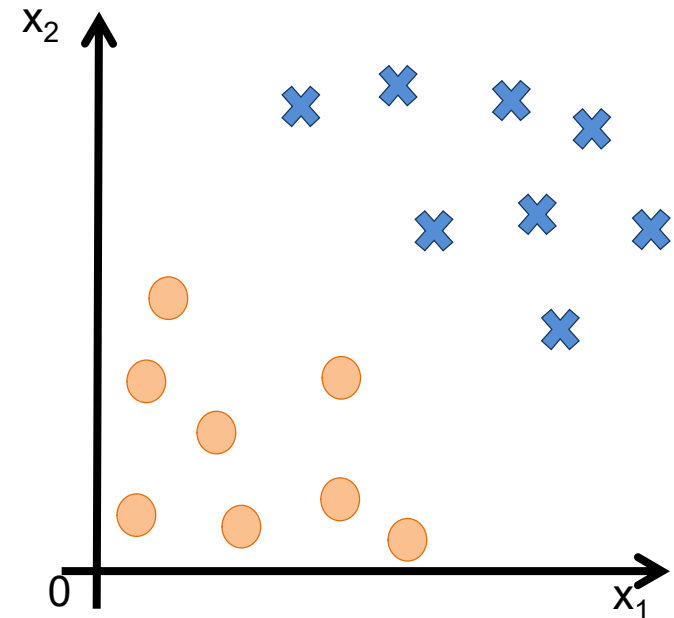
Es un clasificador no paramétrico basado en: un nuevo caso se clasificará con la clase más frecuente a la que pertenecen sus K vecinos mas cercanos.



Algoritmo KNN

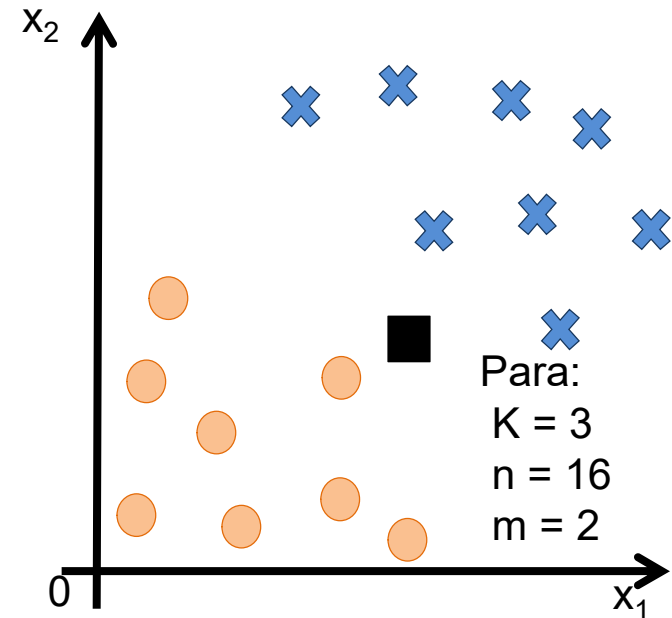
Algoritmo KNN (para test)

- Entrada:
 - Conjunto de datos: $D = (\mathbf{x}^{(1)}, y^{(1)}), \dots, (\mathbf{x}^{(n)}, y^{(n)})$
 - Nuevo caso para clasificar: $\mathbf{x} = (x_1, x_2, \dots, x_m)$
- Salida:
 - Clase que corresponde a $\mathbf{x}$: y
- Pseudocódigo (fase de test) para KNN:
 - PARA todo patrón en D : $(\mathbf{x}^{(i)}, y^{(i)})$
 - Calcular $d_i = d(\mathbf{x}^{(i)}, \mathbf{x})$
 - FIN PARA
 - Ordenar los d_i en orden ascendente.
 - Seleccionar los k primeros casos con menor d_i .
 - Asignar a $\mathbf{x}$ la clase y más frecuente en el subconjunto seleccionado.
 - FIN



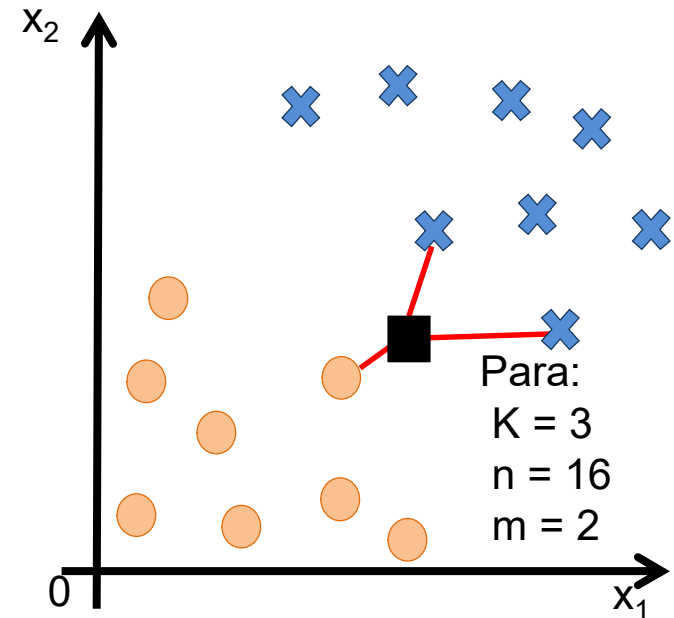
Algoritmo KNN (para test)

- Entrada:
 - Conjunto de datos: $D = (\mathbf{x}^{(1)}, y^{(1)}), \dots, (\mathbf{x}^{(n)}, y^{(n)})$
 - Nuevo caso para clasificar: $\mathbf{x} = (x_1, x_2, \dots, x_m)$
- Salida:
 - Clase que corresponde a $\mathbf{x}$: y
- Pseudocódigo (fase de test) para KNN:
 - PARA todo patrón en D : $(\mathbf{x}^{(i)}, y^{(i)})$
 - Calcular $d_i = d(\mathbf{x}^{(i)}, \mathbf{x})$
 - FIN PARA
 - Ordenar los d_i en orden ascendente.
 - Seleccionar los k primeros casos con menor d_i .
 - Asignar a $\mathbf{x}$ la clase y más frecuente en el subconjunto seleccionado.
 - FIN



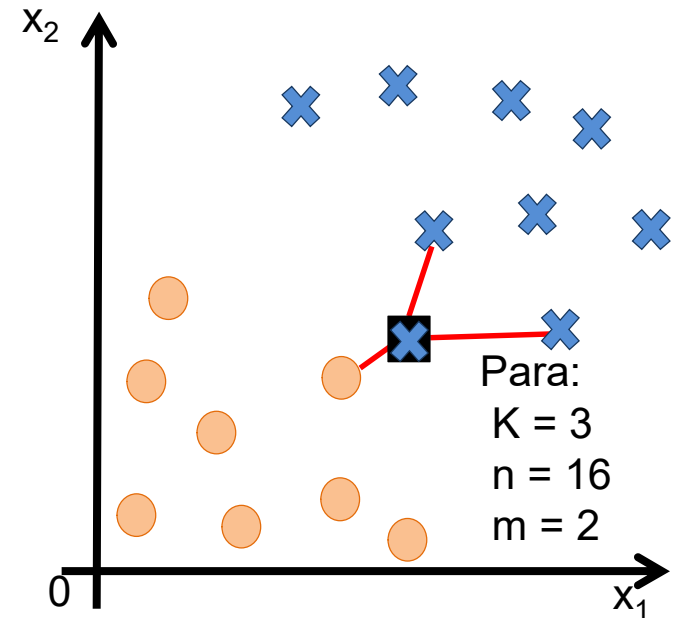
Algoritmo KNN (para test)

- Entrada:
 - Conjunto de datos: $D = (\mathbf{x}^{(1)}, y^{(1)}), \dots, (\mathbf{x}^{(n)}, y^{(n)})$
 - Nuevo caso para clasificar: $\mathbf{x} = (x_1, x_2, \dots, x_m)$
- Salida:
 - Clase que corresponde a $\mathbf{x}$: y
- Pseudocódigo (fase de test) para KNN:
 - PARA todo patrón en D : $(\mathbf{x}^{(i)}, y^{(i)})$
 - Calcular $d_i = d(\mathbf{x}^{(i)}, \mathbf{x})$
 - FIN PARA
 - Ordenar los d_i en orden ascendente.
 - Seleccionar los K primeros casos con menor d_i .
 - Asignar a $\mathbf{x}$ la clase y más frecuente en el subconjunto seleccionado.
 - FIN

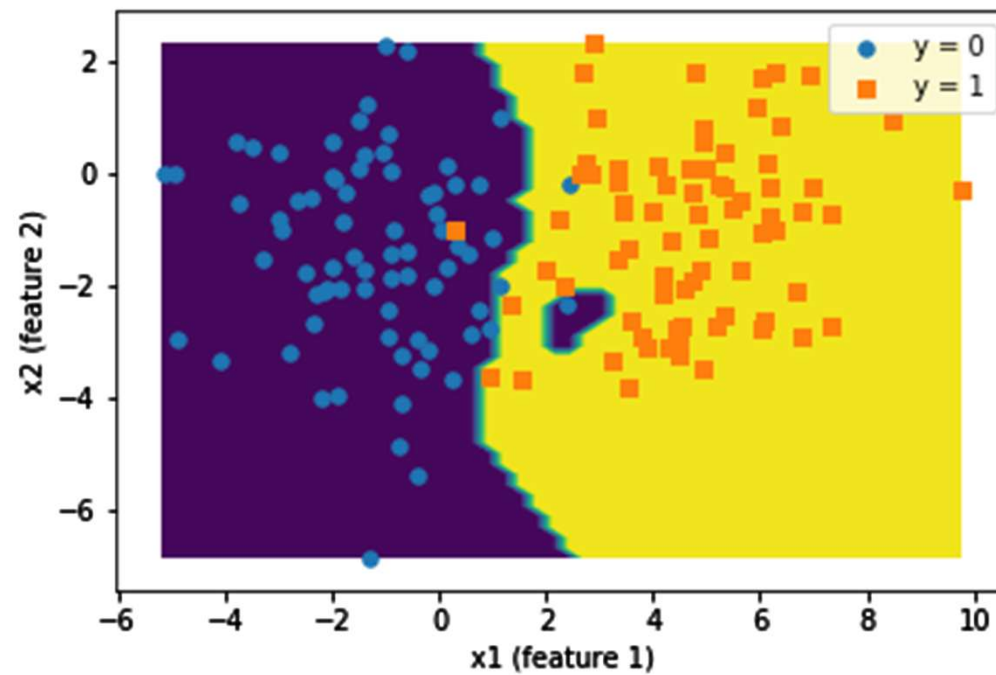


Algoritmo KNN (para test)

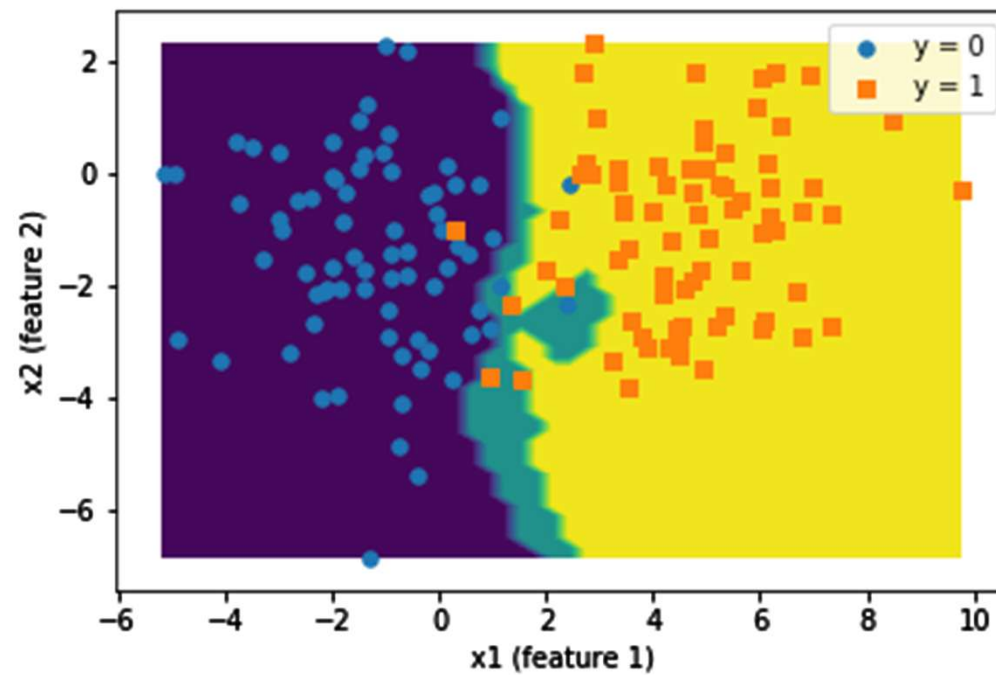
- Entrada:
 - Conjunto de datos: $D = (\mathbf{x}^{(1)}, y^{(1)}), \dots, (\mathbf{x}^{(n)}, y^{(n)})$
 - Nuevo caso para clasificar: $\mathbf{x} = (x_1, x_2, \dots, x_m)$
- Salida:
 - Clase que corresponde a $\mathbf{x}$: y
- Pseudocódigo (fase de test) para KNN:
 - PARA todo patrón en D : $(\mathbf{x}^{(i)}, y^{(i)})$
 - Calcular $d_i = d(\mathbf{x}^{(i)}, \mathbf{x})$
 - FIN PARA
 - Ordenar los d_i en orden ascendente.
 - Seleccionar los K primeros casos con menor d_i .
 - Asignar a $\mathbf{x}$ la clase y más frecuente en el subconjunto seleccionado.
 - FIN



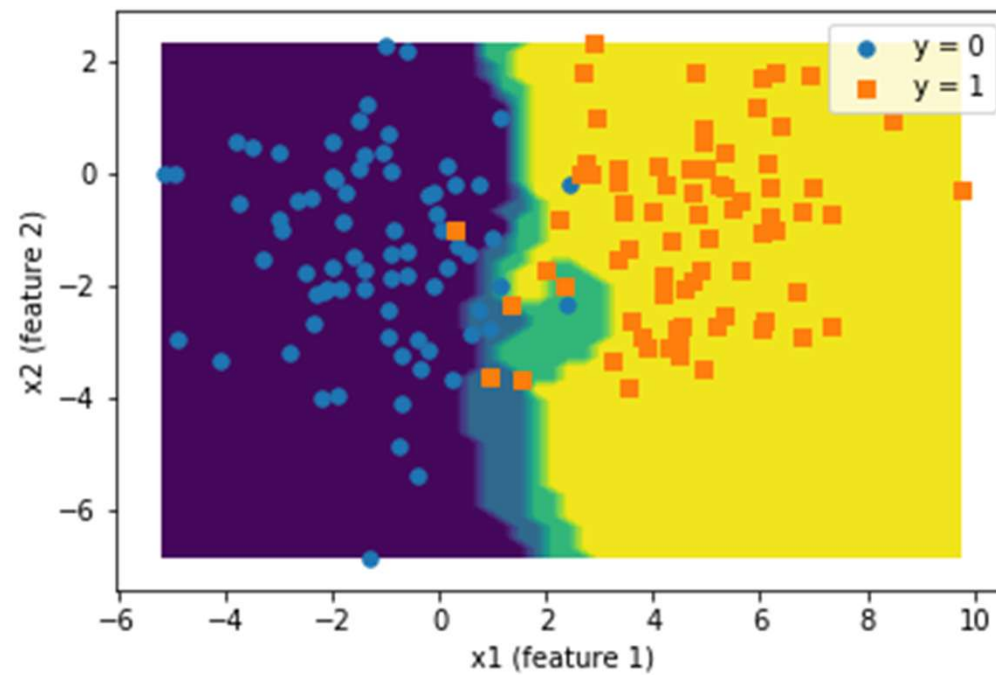
Ejemplo KNN K=1



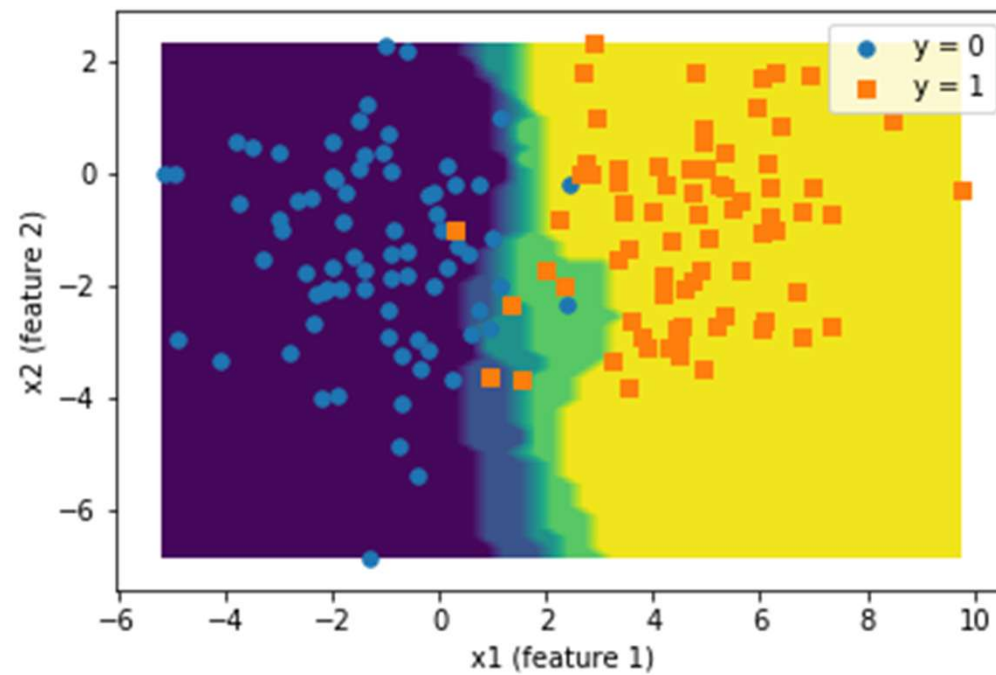
Ejemplo KNN K=2



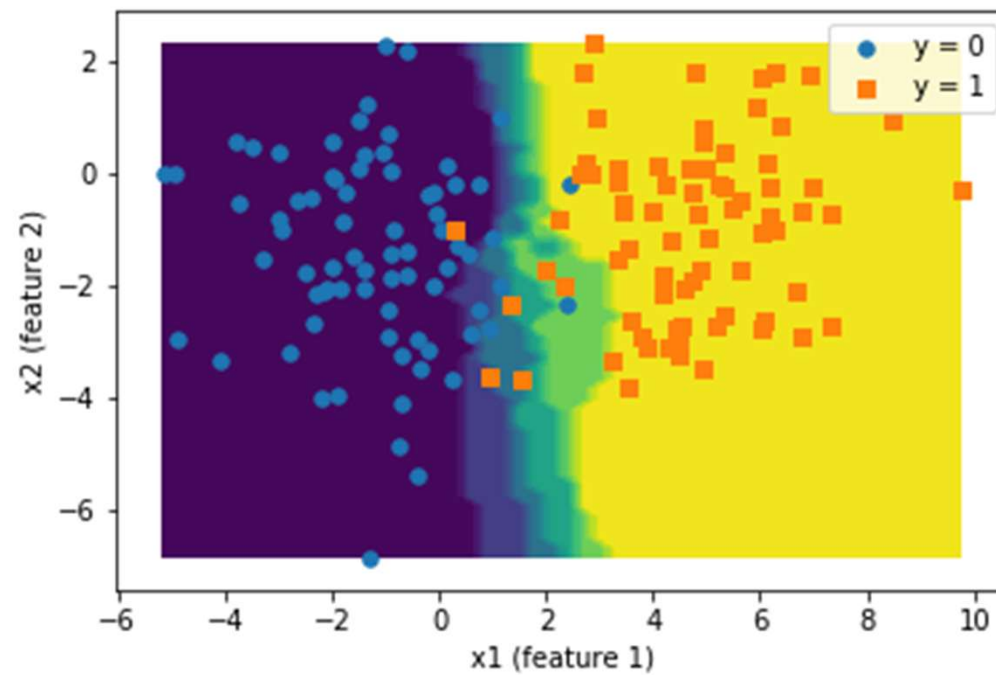
Ejemplo KNN K=3



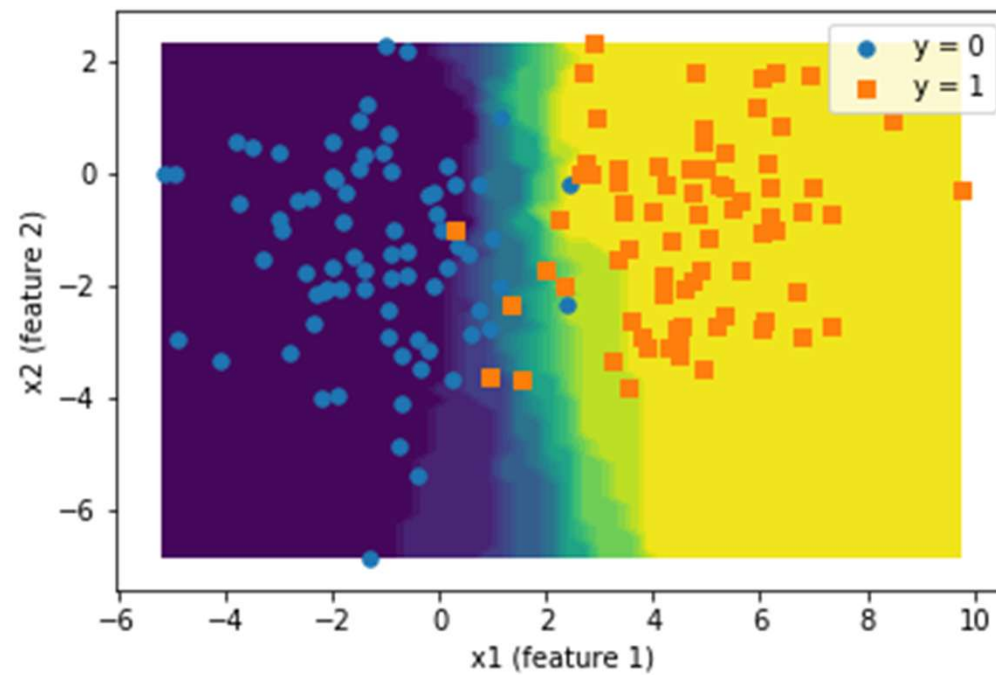
Ejemplo KNN K=4



Ejemplo KNN K=5



Ejemplo KNN K=10

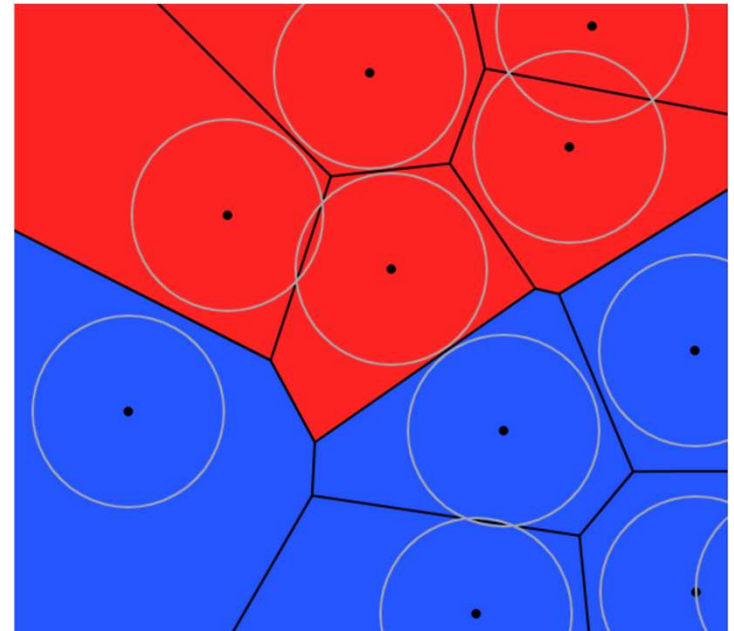


Características de KNN

K Nearest Neighbours (KNN)

Es un clasificador no paramétrico basado en: un nuevo caso se clasificará con la clase más frecuente a la que pertenecen sus k vecinos mas cercanos.

- Con $K = 1$ el clasificador genera **diagramas de Voronoi**.
- La frontera de decisión es **no-lineal**.



K Nearest Neighbours (KNN)

Ventajas:

- Es de fácil implementación.
- Es un procedimiento de clasificación simple pero efectivo.
- Aplica a clasificación multiclase.
- La frontera de decisión es no lineal.
- No hay esfuerzo de entrenamiento (aprendizaje perezoso).
- La calidad de las predicciones mejora con mas datos de entrenamiento.
- Un único parámetro: K.
- Se puede utilizar para regresión.

K Nearest Neighbours (KNN)

Desventajas:

- ¿Que es “cercano”? Se debe especificar una **métrica de distancia**:
 - Ejemplos: Euclídea, Manhattan, Coseno, etc.
- El **costo computacional**: en test, debe almacenar y buscar en el conjunto de entrenamiento entero. Existen alternativas:
 - Árboles k-d: estructuras de datos mas eficientes que particionan el espacio Euclídeo.
 - Locality Sensitivity Hashing: estructura de datos mas eficiente basada en tablas hash.
- No tiene un modelo explícito.
- Si hay muchos datos, el costo computacional sube.
- La dimensionalidad (m) influye mucho en el rendimiento.
- Es especialmente problemático ante la “maldición de la dimensionalidad”: mientras mas dimensiones se tiene, mas ejemplos se necesitan para aprender la hipótesis.
 - Al aumentar las dimensiones, decrece exponencialmente el número de ejemplos contenidos en un volumen determinado.

Considerar: el porcentaje de observaciones bien clasificadas no es monótono en relación a k.

¿Preguntas?

